

Audit Trail

Event-Sourced Inventory & Logistics Ledger — Full Project Description

1. Project Overview

Project: Project 2 – “Audit Trail”

Domain: Supply Chain & FinTech

Primary Architecture: Event Sourcing + CQRS

Backend: Node.js / Express

Database: MongoDB

Frontend: React

Visualization: Recharts

Audit Trail is a logistics and inventory ledger designed to preserve an immutable, chronological history of events. Instead of relying on a traditional CRUD model where an update can overwrite the previous state, the system records each state-changing operation as an event. The current state can then be reconstructed by replaying the historical events.

2. Problem Statement

In a traditional MongoDB CRUD design, updating inventory from 10 to 5 can overwrite the previous value. For highly regulated domains such as logistics and finance, losing the previous state is unacceptable. The system therefore needs an immutable chronological log of every event that contributes to the current state.

3. Main Objective

Objective: Build a logistics ledger where state-changing operations are stored as immutable events, allowing the application to reconstruct the current state and inspect the complete historical timeline of a shipment or container.

4. Example Use Case

A logistics manager searches for a specific shipping container from the Audit Trail dashboard. Rather than simply reading the container’s latest location from MongoDB, the Node.js backend reconstructs its state by replaying an append-only sequence of events.

Example event sequence:

```
CONTAINER_CREATED → LOADED_ON_SHIP → TEMPERATURE_SPIKE → ARRIVED_AT_PORT
```

If a dispute occurs about when a temperature spike happened, the manager can inspect the immutable historical timeline and determine the event sequence.

5. Core Concept — Event Sourcing

Traditional CRUD: action → database update → previous state may be overwritten.

Event Sourcing: action → validate command → create event → append event → preserve history → replay events when state is required.

The event history is central to reconstructing the shipment’s state. The project document describes the backend as mathematically “folding” events to project the current state.

6. Architecture

Component	Responsibility
Event Store — MongoDB	Append-only storage for raw events.

Component	Responsibility
Event Sourcing Engine — Node.js	Accepts commands, validates them, appends events, and reconstructs state by replaying/folding events.
CQRS	Separates write operations (Commands) from read operations (Queries).
Forensic Dashboard — React	Provides shipment search, historical timelines, reconstructed states, and temporal views.
Recharts	Visualizes sensor metrics such as temperature fluctuations alongside the event timeline.

7. Event Store Design

The Week 2 specification defines the append-only event log around these fields:

- `aggregateId` — identifies the shipment/container aggregate.
- `eventType` — identifies what happened.
- `payload` — contains event-specific data.
- `timestamp` — records when the event occurred.
- `version` — supports ordered event history and concurrency control.

The Event Store must be treated as historical source data rather than a normal CRUD collection. The Mid-Project Review specifically requires proof that UPDATE and DELETE operations are prevented.

8. CQRS

CQRS means Command Query Responsibility Segregation. The project separates API routes that write data from routes that read data.

Example specified routes:

```
POST /shipment/move → Command / write operation
GET /shipment/:id → Query / read operation
```

9. End-to-End Workflow

1. A user performs a shipment/container action.
2. The React application sends a command to the Node.js/Express backend.
3. The backend validates the command.
4. A new event is created.
5. The event is appended to MongoDB's Event Store.
6. Historical events remain unchanged.
7. When the current state is needed, the backend replays/folds the relevant events.
8. The reconstructed state is returned to the query side/dashboard.
9. In Week 3, a background worker also updates an optimized Read Model from new events.
10. The dashboard visualizes the event timeline, historical state, and sensor metrics.

10. Week-Wise Development Plan

Week / Stage	Backend Architecture	Frontend Visualization
Week 1	CQRS Setup: Structure the Express backend using CQRS principles. Create separate routers for Commands and Queries. Example: POST /shipment/move and GET /shipment/:id.	Dashboard Scaffolding: Build the React UI with a search bar for specific shipment IDs.
Week 2	Event Store: Design the MongoDB append-only event log with aggregateId, eventType, payload, timestamp, and version.	Timeline UI: Build a vertical React timeline showing the raw sequence of events for a shipment.
Mid-Project Review	Immutability Audit: Prove that UPDATE and DELETE operations are prevented on the Event Store.	Reconstruction Check: Ensure the backend correctly calculates shipment state by replaying historical events.
Week 3	Projections / Read Models: Build a background Node.js worker that listens for new events and updates a separate optimized MongoDB Read Model.	State Scrubbing: Build a UI slider allowing users to rewind time and view the shipment's state as it existed in the past, including the specified three-day example.
Week 4	Concurrency Control: Implement Optimistic Concurrency Control (OCC) and reject commands when the stored event version has changed since the user loaded the page.	Refine & Polish: Integrate Recharts to visualize sensor metrics such as temperature fluctuations over the event timeline.
Final Review	Demonstrate the move from basic CRUD to enterprise-oriented Event Sourcing and the completed backend architecture.	Demonstrate a specialized logistics dashboard capable of complex temporal analysis.

11. Week 1 — Detailed Understanding

Backend goals:

- Establish the Express backend structure around CQRS.
- Separate Command routes from Query routes.
- Provide the initial shipment movement command and shipment query flow.

Frontend goals:

- Create the initial React dashboard.
- Provide a search mechanism for shipment IDs.

Testing focus:

- Verify Command and Query routes are separated correctly.
- Test valid and invalid shipment IDs.
- Test valid and invalid command requests.
- Verify expected HTTP responses and basic error handling.

12. Week 2 — Detailed Understanding

Backend goals:

- Implement the append-only MongoDB event schema.
- Store aggregateId, eventType, payload, timestamp, and version.
- Ensure events represent the historical sequence rather than replacing previous state.

Frontend goals:

- Display events in chronological order.

- Create a vertical timeline for a shipment.

Testing focus:

- Verify event fields are stored correctly.
- Verify event ordering and timestamps.
- Verify historical events remain available after new events are added.
- Perform the required immutability audit against UPDATE and DELETE behavior.

13. Mid-Project Review — Critical Validation

- **Immutability Audit:** demonstrate that historical Event Store records cannot be updated or deleted.
- **Reconstruction Check:** provide a known sequence of events and verify that replaying them produces the expected current state.
- Verify the dashboard timeline corresponds to the stored event sequence.

14. Week 3 — Detailed Understanding

- **Read Model:** Create a separate optimized MongoDB collection for read operations.
- **Background Worker:** Listen for new events and update the Read Model.
- **State Scrubbing:** Allow users to move through time and inspect a shipment's historical state.
- **Testing:** Verify event-to-read-model propagation, read-model correctness, and historical state reconstruction at different points in time.

15. Week 4 — Detailed Understanding

- **Optimistic Concurrency Control:** Reject a command if the stored event version has changed since the user's earlier read.
- **Visualization:** Use Recharts to display sensor metrics such as temperature fluctuations alongside the event timeline.
- **Testing:** Simulate concurrent updates, verify stale-version rejection, and validate the correlation between sensor data and events.

16. Major Features

- Immutable chronological event history.
- Event-sourced shipment/container state reconstruction.
- CQRS-based separation of Commands and Queries.
- MongoDB append-only Event Store.
- Historical event timeline.
- Forensic dashboard for investigating shipment history.
- Read Model projections for optimized reads.
- Time-based state scrubbing / historical state viewing.
- Optimistic Concurrency Control.
- Sensor metric visualization using Recharts.
- Historical analysis of logistics events.

17. Testing Areas Relevant to the Project

Testing Area	What should be verified
Functional Testing	Commands, queries, shipment search, timeline, state reconstruction, and dashboard behavior.
Event Store Testing	Correct event structure, timestamps, versions, and append-only behavior.
Immutability Testing	Historical events cannot be updated or deleted.
Event Replay Testing	Known event sequences reconstruct the expected current state.
CQRS Testing	Command and Query responsibilities remain separated.
Read Model Testing	New events produce the expected Read Model changes.
Time Scrubbing Testing	Historical states match the event history at selected points in time.
Concurrency Testing	Stale event versions are rejected by OCC.
Visualization Testing	Timeline order and sensor charts correctly represent underlying data.
Regression Testing	New changes do not break previously validated event, query, or visualization behavior.

18. Documentation Areas

- Project overview and business problem.
- System architecture and component responsibilities.
- Event Sourcing explanation.
- CQRS explanation and Command/Query route separation.
- MongoDB Event Store schema.
- Event lifecycle and state reconstruction workflow.
- Read Model and projection workflow.
- State scrubbing / temporal analysis behavior.
- Optimistic Concurrency Control behavior.
- API documentation for implemented endpoints.

- Testing strategy, test cases, defects, and test results.
- Known limitations and future improvements.

19. Important Architectural Rules

- **Events are historical records:** they should not be treated like ordinary mutable CRUD records.
- **Commands write:** write operations belong to the Command side of CQRS.
- **Queries read:** read operations belong to the Query side.
- **State can be reconstructed:** the system must be able to derive state by replaying events.
- **Versions matter:** the event version supports concurrency control.
- **Read performance can be optimized:** projections/read models are introduced in Week 3.

20. Final Project Outcome

The final system is intended to demonstrate an enterprise-oriented alternative to basic CRUD for logistics and inventory. Its key capability is not simply storing the latest shipment status, but preserving and using the complete sequence of events that produced that status. The backend provides event sourcing, CQRS, projections, and concurrency control, while the React dashboard provides chronological and temporal analysis of shipment history and sensor information.

21. One-Line Project Summary

Audit Trail is an event-sourced logistics ledger that permanently records shipment events, reconstructs current and historical states, and provides a forensic dashboard for analyzing the complete timeline of inventory and container activity.